

Custos / Clew 프로젝트 종합 기록

멀티에이전트 트레이스 낭비 탐지 — 지식 정리 · 진행 현황 · 관련 논문

작성일 2026-06-12 · 회사 Custos · 제품 Clew · 개발 도구 Claude Code

이 문서는 지금까지 진행한 작업 전체를 한 번에 복원할 수 있도록 정리한 자기완결 기록입니다. (1) 이 프로젝트가 무엇이고 왜 이렇게 설계했는지, (2) 알아야 할 핵심 지식과 개념, (3) 지금 어디까지 왔는지, (4) 기반이 된 관련 논문 — 네 가지를 모두 담았고, 각 부분마다 이해를 돕는 쉬운 설명을 붙였습니다.

0. 한눈에 보기

Custos(회사)가 만드는 Clew(제품)는 여러 AI 에이전트가 협업하는 시스템의 실행 기록(트레이스)을 받아 **헛도는 반복과 중복된 핸드오프**를 찾아 그로 인해 낭비되는 토큰(=돈)을 리포트하는 도구입니다. 출발점은 뼈아픈 교훈이었습니다. 이전 검증 실험(v1)이 '실패를 예측한다'는 신호를 만들었지만, 알고 보니 그 신호는 사실상 '트레이스 길이'를 재고 있었고 정상적인 수렴을 정체로 오판했습니다. 그래서 이번 빌드의 제1원칙은 '**코딩 에이전트(와 우리 자신)가 스스로를 속이지 못하게 판을 먼저 짠다**'가 되었습니다.

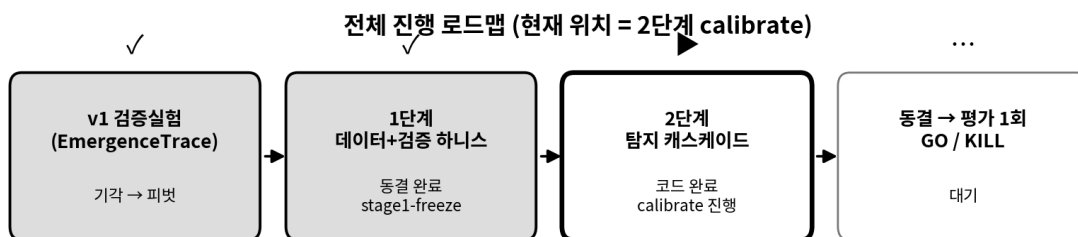


그림 0. 전체 진행 로드맵. v1 기각 → 1단계 동결 완료 → 2단계 탐지기 구현(현재 calibrate) → 동결·평가 대기.

1. 무엇을 만드는가 — 문제와 제품

쉬운 설명

여러 AI 에이전트가 한 작업을 나눠서 처리할 때, 실패의 상당수는 '모델이 멍청해서'가 아니라 **에이전트 사이의 문제**에서 옵니다. 같은 일을 두 번 하고, 이미 가진 정보를 또 조회하고, 두 에이전트가 같은 결론을 주고받으며 헛돕니다. 이런 낭비는 에러를 내지 않고 조용히 토큰만 태웁니다. 기존 관측 도구는 대부분 '단일 에이전트'용이라 이 '사이의 층'을 못 봅니다. Clew는 바로 그 낭비를 **보이게 만들고(→ 결국 막아)** 신뢰성과 비용을 지키는 것을 목표로 합니다.

핵심 wedge (첫 진입점)

‘실패를 예측한다’(모호하고 v1에서 실패)가 아니라 ‘멀티에이전트가 중복 루프·핸드오프로 태우는 토큰 낭비를 탐지·차단한다’입니다. 측정 가능하고, 비용이라는 분명한 언어로 말할 수 있으며, 멀티에이전트에 특화돼 있습니다.

우리가 다루는 네 가지 낭비 패턴

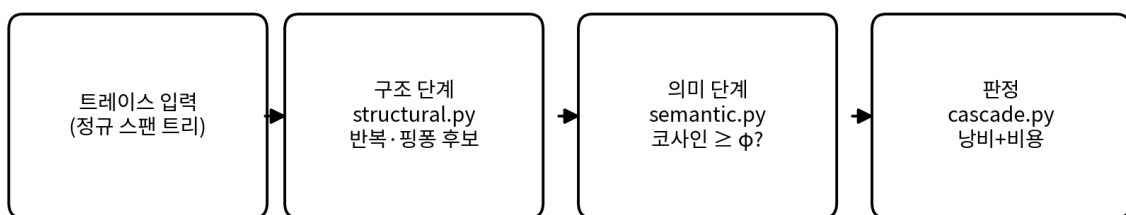
패턴	무엇인가	예시
repeat_node	같은 노드를 반복 호출하며 같은 작업 재생산	analyze를 3번, 출력이 거의 동일
regen_handoff	넘겨받은 에이전트가 앞 요약의 거의 재생성	B가 A의 요약을 표현만 바꿔 다시 씀
pingpong_aba	두 에이전트가 결론 없이 주고받기 반복	A→B→A→B, 2회차가 1회차 반복
requery_known	이미 가진 정보를 다시 조회	같은 키를 두 번 lookup

2. 당신이 알아야 할 핵심 지식

2.1 캐스케이드(구조 → 의미)가 핵심 아이디어

탐지의 심장은 두 단계 캐스케이드입니다. 먼저 **구조 단계**가 ‘같은 노드가 반복됐다 / 핑퐁이 있다’처럼 값싸고 빠른 신호로 **의심 후보를 넓게** 수립합니다. 그다음 **의미 단계**가 그 후보들의 출력 텍스트를 임베딩해 코사인 유사도가 임계값 ϕ 이상이면 ‘진짜 중복’으로 확정합니다. 둘 중 하나만으로는 약합니다 — 구조만 보면 정상 반복까지 낭비로 오인하고, 의미만 보면 비싸고 산만합니다. **둘을 직렬로 결합**하는 것이 핵심입니다.

캐스케이드 탐지 구조 (구조 → 의미 → 판정)



구조는 후보를 넓게 던지고 → 의미가 정밀하게 거른다

그림 1. 캐스케이드 탐지 구조. 구조가 후보를 넓게 던지고, 의미(코사인 $\geq \phi$)가 정밀하게 거른다.

왜 중요한가: v1의 실수 중 하나가 두 신호를 ‘합(sum)’으로 섞은 것이었습니다. 올바른 방식은 합이 아니라 **직렬 게이트(구조가 후보를 만들고 → 의미가 거름)**입니다. 이는 IBM 연구(3장)가 실증적으로 보인 구조이기도 합니다.

2.2 자기기만을 막는 규율 — 이 프로젝트의 영혼

v1의 교훈은 '신호가 그럴듯해 보였지만 사실은 엉뚱한 것(길이)을 재고 있었다'였습니다. 그래서 우리는 다음 규율을 코드로 강제합니다(말로 지키겠다는 약속이 아니라).

- **기준 사전 동결:** 성공/중단 기준(GO/KILL)을 라벨·결과를 보기 전에 적고 git 에 박제. 결과 보고 기준을 바꾸면 검증 무효.
- **누수 차단:** 탐지 코드는 정답 라벨을 절대 읽지 못함. 여기면 빌드가 깨지도록 정적(AST)·런타임 가드로 강제.
- **dev/eval 분리:** 파라미터(ϕ -N·모델)는 dev set(seed=7)에서만 정하고, 진짜 평가 set(seed=42)은 단 한 번만 측정.
- **분포 분리로 임계 결정:** ϕ 를 'dev 에서 F1 최대화'로 정하지 않음(과적합). 두 분포의 겹으로 정함.
- **외부 연구와 우리 결과 구분:** IBM 의 F1 0.72 는 그들의 결과이지 우리 것이 아님. 과장 금지.

2.3 검증 데이터를 정직하게 만드는 법 — Paired Design

탐지기를 시험하려면 '정답이 박힌' 트레이스가 필요합니다. 핵심은 **positive(낭비)**와 **clean(정상)** 쌍을 같은 구조로 만드는 것입니다. 만약 clean이 구조적으로 '평평'(반복 없음)하면, '반복=낭비'라는 구조 단독 탐지기만으로도 정답을 맞혀버려 — 의미 단계가 일을 안 했는데도 성공한 것처럼 보입니다(v1의 2차 함정). 그래서 clean 트윈은 positive와 같은 반복·핸드오프 토폴로지를 갖되, 차이는 오직 '반복된 작업이 의미적으로 중복인가' 하나여야 합니다. 그래야 의미 단계가 실제로 판정자가 됩니다.

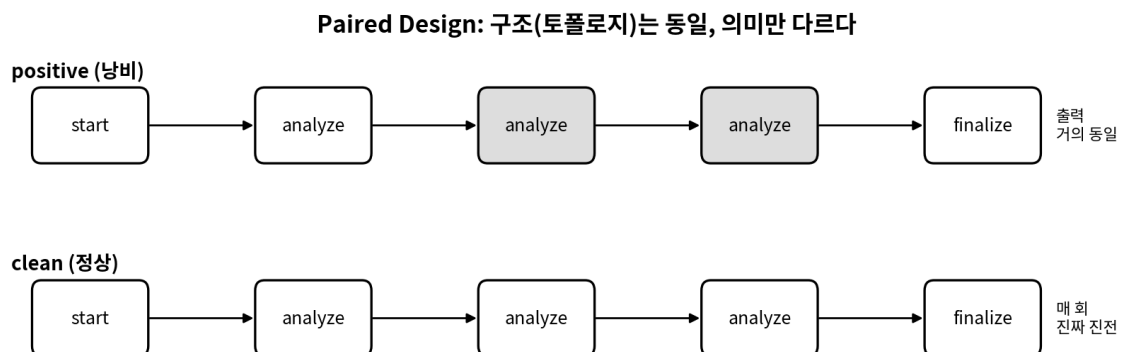


그림 2. Paired Design. 위(positive)와 아래(clean)는 노드 시퀀스가 동일하고, 회색 칸(낭비)의 의미적 중복 여부만 다르다.

3. 기반이 된 관련 논문

3.1 IBM Research — Unsupervised Cycle Detection in Agentic Applications

arXiv:2511.10650 (George 외, IBM Research, 2025). 우리 캐스케이드 설계의 직접적 근거.

이 논문은 LLM 에이전트 실행에서 조용히 자원을 태우는 '숨은 사이클(반복)'을 찾는 비지도 방법을 제안합니다. 먼저 시간적 호출스택 분석으로 명시적 루프를 찾고, 그다음 의미 유사도(출력 간 코사인)로 '중복 내용 생성'이라는 미묘한 사이클을 잡습니다. LangGraph 기반 주식 애플리케이션의 1,575개 트레이스에서 결합 방식이 F1 0.72(정밀도 0.62, 재현율 0.86)를 달성했고, 이는 구조 단독(F1 0.08)이나 의미 단독(F1 0.28)을 크게 앞섰습니다.

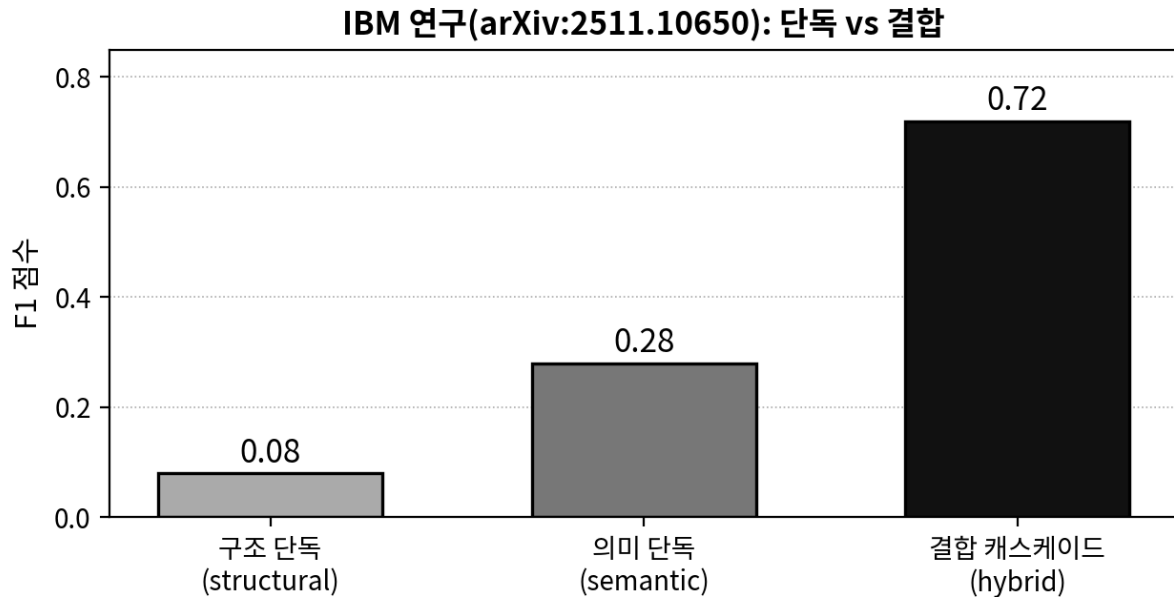


그림 3. IBM 연구의 핵심 수치. 단독 방법은 약하고, 구조+의미 결합 캐스케이드가 F1 0.72로 압도한다.

꼭 기억할 것: F1 0.72는 IBM의 데이터·과제에서 나온 그들의 결과입니다. 우리 데이터에서의 성능은 우리가 직접 검증해야 하며, 이 구분을 모든 공개 커뮤니케이션에서 유지해야 합니다.

3.2 MAST — Why Do Multi-Agent LLM Systems Fail?

arXiv:2503.13657 (Cemri 외, UC Berkeley, NeurIPS 2025). 시장·문제 정의의 근거.

7개 멀티에이전트 프레임워크의 1,600여 개 주식 트레이스를 분석해, 멀티에이전트 실패를 **14개 모드**, **3개 범주**로 분류한 최초의 실증적 분류 체계(MAST)입니다. 세 범주는 (i) 시스템 설계 문제, (ii) 에이전트 간 불일치, (iii) 작업 검증입니다. 주식자 간 일치도(Cohen's κ)는 0.88로 높았습니다. 이 분류에서 '단계 반복(Step Repetition)'이 사이클의 증상으로 명시되며, 실패의 상당 부분이 '모델 능력'이 아니라 '에이전트 사이'의 문제임을 보여줍니다 — Clew의 문제의식과 정확히 맞닿습니다.

MAST의 또 다른 함의: '기반 모델이 좋아지는 것만으로는 이 실패들을 다 해결할 수 없다'고 결론짓습니다. 즉 더 똑똑한 모델을 기다리는 대신, 사이·조정·검증을 직접 관측·교정하는 도구가 필요하다는 뜻입니다.

4. 지금 어디까지 왔는가

4.1 1 단계 — 데이터 기반 + 검증 하니스 (완료·동결)

탐지 로직을 만들기 전에, 정직하게 검증할 '판'부터 깔았습니다. 입력 포맷은 표준인 OpenInference/OTel 스펙으로, 첫 어댑터는 LangGraph로 정했습니다. 만든 것은 (1) 정규 스펙 트리 모델(output_text 필수), (2) LangGraph 어댑터, (3) paired 검증 라벨셋(4패턴 × 10쌍, positive 40 + clean 40), (4) 라벨 보기 전 동결한 성공/중단 기준, (5) 누수 가드입니다.

도중에 두 가지 결함을 잡았습니다. 처음엔 positive의 중복 출력이 '바이트 단위로 동일'해 너무 쉬웠고(단순 문자열 일치로 통과 가능), 이를 '뜻은 같고 글자는 다른' 현실적 패러프레이즈로 고쳤습니다(단, requery는 같은 키 재조회라 동일 출력이 정상이라 그대로 둠). 또 manifest 해시가 트레이스 본문 변경을 반영하지 못하던 버그를 고쳐, 동결이 본문까지 박제하도록 강화했습니다.

동결 결과: commit 0fa25e0, 태그 stage1-freeze, 테스트 92/92 통과.

4.2 누수 차단과 dev/eval 분리 — 코드로 강제

탐지 코드(src/clew/detect/*)는 라벨 파일을 import하거나 읽으면 빌드가 깨집니다. 라벨을 읽는 것은 평가 스크립트(evaluate.py) 단 하나뿐이며, 탐지기에는 라벨을 인자로도 넘기지 않습니다. 파라미터는 dev set(seed=7)에서만 정하고, evaluate는 CRITERIA에 파라미터가 '미정(TBD)'이면 평가 set을 읽기도 전에 오류를 내 막습니다. 동결 전에는 진짜 평가 데이터를 물리적으로 만질 수 없습니다.

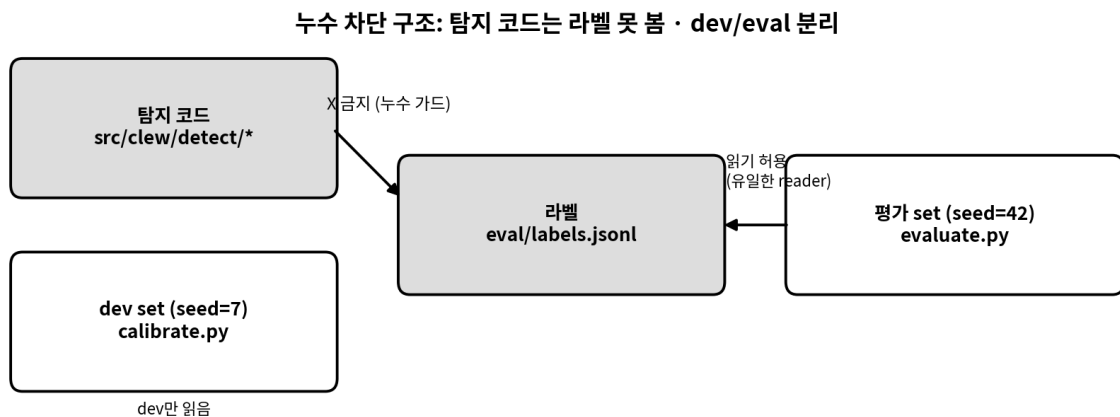


그림 4. 누수 차단 구조. 탐지 코드는 라벨 접근 금지(X), evaluate만 유일하게 라벨을 읽으며, calibrate는 dev만 본다.

4.3 2 단계 — 탐지 캐스케이드 (코드 완료, calibrate 진행 중)

structural.py(후보 탐지), semantic.py(임베딩+코사인, 모델 버전 고정·캐시로 결정론), cascade.py(구조 AND 의미 → 낭비+비용), evaluate.py(평가 1회·GO/KILL 판정)까지 코드를 완성했습니다. 결정적 진전은 calibrate를 분포 분리 기반으로 다시 설계한 것입니다. ϕ 는 'dev 라벨로 F1을 최대화'(과적합)하지 않고, 중복 쌍과 진전 쌍의 코사인 두 분포의 겹 중점에 둡니다. 그리고 dev에서 두 분포가 안 갈리면 ($\text{gap} \leq 0$, Cohen's $d < 0.5$, dev FPR 추정 > 0.15) 평가 전에 멈추는 가드를 넣었습니다.

4.4 현재 막힌 지점 — 그리고 이것이 왜 좋은 신호인가

1차 모델(다국어 MiniLM)로 calibrate를 돌리자 **분리 가드가 FAIL**했습니다. 평균은 잘 같렸지만(중복 0.82 vs 진전 0.43, Cohen's d 2.85), **진전 분포의 꼬리**가 중복 분포 안쪽까지 침투해 dev FPR 추정이 0.20(기준 0.10의 2배)이 됐습니다.

분포	개수	P10	중앙값	P90	평균
중복(dup)	50	0.625	0.834	1.000	0.816
진전(prog)	50	0.338	0.365	0.678	0.429

분리 가드	임계	실제값	판정
gap (P10 dup – P90 prog) > 0	> 0	-0.053	FAIL
Cohen's d ≥ 0.5	≥ 0.5	2.855	PASS
dev FPR 추정 ≤ 0.15	≤ 0.15	0.20	FAIL

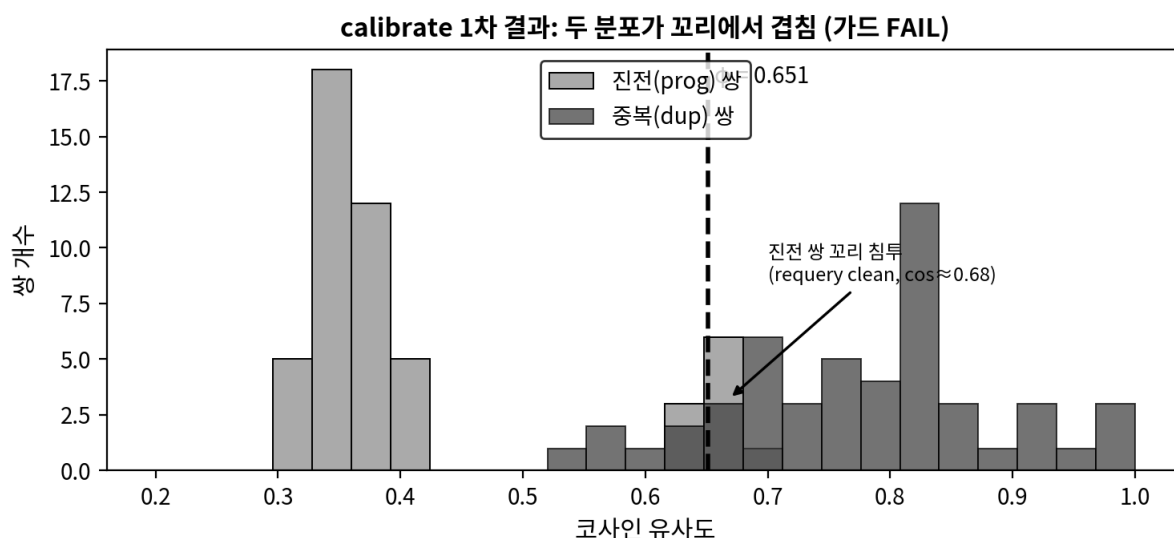


그림 5. calibrate 1차 결과. 대부분 잘 갈리지만 진전 쌍 일부(requery clean)가 꼬리에서 중복 영역으로 침투해 가드가 막았다.

이것은 시스템이 의도대로 작동한 것입니다. v1 때는 이런 신호를 못 보고 '된다'고 착각했지만, 이번엔 진짜 평가 데이터를 만지기도 전에 dev에서 '이 설정으로는 중복과 진전이 깨끗이 안 갈린다'를 잡아냈습니다. 평가 set은 아직 한 번도 건드리지 않았으므로, 지금 방향을 바꿔도 규율은 깨끗합니다 — 이것이 dev/eval 분리를 갈아둔 정확한 이유입니다.

4.5 원인 진단 — 데이터 결함으로 확정

코사인이 가장 높은 진전 쌍 상위 8개를 출력해 원인을 특정했습니다. 침투 10개가 **전부 requery_known clean** 한 패턴에 몰려 있었고(다른 세 패턴은 침투 0), 원문은 다음과 같았습니다.

```
origin      : name=Alice, plan=Pro, MRR=$59
candidate   : name=Bob, plan=Free, MRR=$0
cosine      : 0.678
```

두 레코드는 사람·플랜·금액이 모두 다른 **명백히 별개의 내용**인데, 임베딩이 'name=..., plan=..., MRR=\$...'라는 **정형 스키마의 표면 형식**을 의미 유사도로 오인했습니다. 게다가 그 동일 쌍이 10개 트레이스에 그대로 복제돼(다양성 0) 있었습니다. 즉 모델의 한계가 아니라(다른 세 패턴은 이 모델로 잘 걸림) 생성기가 **requery clean**을 비현실적으로 **확일적·정형적으로 만든 데이터 결함**입니다.

4.6 다음 행동

1. **requery_known clean** 생성기 수정: 두 번째 조회가 의미적으로 분명히 다른 내용이 되도록, 정형 스키마 표면이 코사인을 띄우지 않게 다양성·자연어를 부여(positive 의 동일-키 재조회는 그대로).
2. 구조 토폴로지 동등성과 hint-word 금지 등 기존 규율·테스트는 유지.
3. dev set 재생성 → calibrate 재실행 → 세 가드($\text{gap} > 0$, $d \geq 0.5$, $\text{dev FPR} \leq 0.15$) 통과 확인.
4. 통과 시: 같은 결함이 평가 set(seed=42)에도 있으므로 평가 set 도 재생성·재동결(라벨셋 변경 → 1 단계 재동결 또는 CRITERIA 갱신).
5. 그 다음에야 파라미터 동결 → 평가 단 1 회 → GO/KILL 판정.

주의: 이번 수정은 '평가 결과를 보고 데이터를 유리하게 바꾸는 것'이 아니라 'dev 분리 실패로 드러난 생성기 결함을 고치는 것'입니다. 평가 set 결과는 아직 본 적이 없으므로 규율 위반이 아닙니다. 단, 수정 후에도 정형 출력에서 임베딩이 약하면, 그건 데이터가 아니라 '우리 신호가 정형 스키마 재조회에는 약하다'는 진짜 한계 신호이며, 그땐 구조+인자 비교로 신호를 보강해야 합니다.

5. 동결된 성공/중단 기준 (참고)

구분	조건	의미
GO (통과)	$F1 \geq 0.80$ AND control FPR ≤ 0.10	낭비를 잘 잡고, 정상을 거의 오탐 안 함
KILL (중단)	$F1 < 0.60$ OR control FPR > 0.25	신호 실패 → 재설계로 회귀
회색지대	그 사이 (예산 N=3회)	집계 지표만 보며 제한적 재조정

control FPR(정상 트레이스를 낭비로 오판하는 비율)이 가장 중요한 가드입니다. 이것이 바로 v1이 무너진 지점 — '정상 수렴을 정체로 오판'한 실수 — 이기 때문입니다.

핵심 요약: 1단계(검증 하니스)는 완료·동결, 2단계 탐지기 코드는 완성, 현재 calibrate에서 분리 가드가 정직하게 발동해 원인을 'requery_known 데이터 결함'으로 확인한 상태입니다. 평가 set은 아직 한 번도 건드리지 않았습니다. 다음은 생성기 수정 → dev 재검증 → (통과 시) 평가 set 재동결 → 평가 단 1회로 GO/KILL을 가리는 것이며, 그것이 v1 이후 '우리 데이터에서 신호가 진짜인가'에 대한 첫 정직한 답이 됩니다.